SIMATS ENGINEERING

ASSESSMENT TOOL 2: Schema Analysis and Viva Voce

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO2: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Assessment Tool Weightage: 35%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 20**
Question Count: 2

Code of Conduct

I **Yarra Pawan (192424294)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Y. Pawan**

Database Design and Connectivity Rubric

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Questions:

Set A

Question 1: Student Course Registration Database (10 Marks)

A university maintains the following database schema:

- Student (Student_ID, Student_Name, Department, Year)
- Course (Course_ID, Course_Name, Credits)
- Enrollment (Enrollment_ID, Student_ID, Course_ID, Semester)

Here, Student_ID in the Enrollment table is a foreign key referencing the Student table, and Course_ID is a foreign key referencing the Course table.

Analyze the above schema and write a Python program using SQLite/MySQL/PostgreSQL to connect to the database and retrieve the names of all students along with the courses they are enrolled in during the "Odd 2026" semester using an appropriate SQL JOIN query.

Question 2: Hospital Management Database (10 Marks)

A hospital maintains the following database schema:

- Doctor (Doctor_ID, Doctor_Name, Specialization)
- Patient (Patient_ID, Patient_Name, Age)
- Appointment (Appointment_ID, Patient_ID, Doctor_ID, Appointment_Date)

Analyze the schema and write a Python program to connect to the database and display the patient names, doctor names, and appointment dates for all appointments scheduled after 1 January 2026.

Question 3: Online Shopping Database (10 Marks)

An e-commerce company uses the following schema:

- Customer (Customer_ID, Customer_Name, City)
- Product (Product_ID, Product_Name, Price)
- Orders (Order_ID, Customer_ID, Product_ID, Quantity, Order_Date)

Analyze the schema and write a Python program to retrieve the customer name, product name, quantity

Question 4: Hospital Management Database (10 Marks)

A hospital maintains the following database schema:

- Doctor (Doctor_ID, Doctor_Name, Specialization)
- Patient (Patient_ID, Patient_Name, Age)
- Appointment (Appointment_ID, Patient_ID, Doctor_ID, Appointment_Date)

Analyze the schema and write a Python program to retrieve the doctor name, patient name, and appointment date for all appointments scheduled during the current month.

Question 5: University Course Registration Database (10 Marks)

A university maintains the following database schema:

- Student (Student_ID, Student_Name, Department)
- Course (Course_ID, Course_Name, Credits)
- Enrollment (Enrollment_ID, Student_ID, Course_ID, Semester)

Analyze the schema and write a Python program to retrieve the student name, course name, and semester for all students enrolled in courses carrying more than 3 credits.

Set B

Question 6: Banking Database (10 Marks)

A bank maintains the following database schema:

- Customer (Customer_ID, Customer_Name, City)
- Account (Account_No, Customer_ID, Account_Type, Balance)
- Transaction (Transaction_ID, Account_No, Transaction_Date, Amount)

Analyze the schema and write a Python program to retrieve the customer name, account number, and balance for customers whose account balance exceeds ₹1,00,000.

Question 7: Library Management Database (10 Marks)

A library maintains the following database schema:

- Member (Member_ID, Member_Name, Phone)
- Book (Book_ID, Book_Title, Author)
- Issue (Issue_ID, Member_ID, Book_ID, Issue_Date)

Analyze the schema and write a Python program to retrieve the member name, book title, and issue date for all books issued after 1 January 2026.

Question 8: Hotel Reservation Database (10 Marks)

A hotel maintains the following database schema:

- Customer (Customer_ID, Customer_Name, City)
- Room (Room_ID, Room_Type, Rent)
- Reservation (Reservation_ID, Customer_ID, Room_ID, Check_In_Date, Check_Out_Date)

Analyze the schema and write a Python program to retrieve the customer name, room type, and check-in date for customers who have reserved Deluxe rooms.

Question 9: Employee Project Management Database (10 Marks)

A software company maintains the following database schema:

- Employee (Employee_ID, Employee_Name, Department)
- Project (Project_ID, Project_Name, Budget)
- Works_On (Employee_ID, Project_ID, Role)

Analyze the schema and write a Python program to retrieve the employee name, project name, and role of employees working on projects with a budget greater than ₹50,00,000.

Question 10: Airline Reservation Database (10 Marks)

An airline company maintains the following database schema:

- Passenger (Passenger_ID, Passenger_Name, City)
- Flight (Flight_ID, Flight_Name, Destination)
- Booking (Booking_ID, Passenger_ID, Flight_ID, Travel_Date)

Analyze the schema and write a Python program to retrieve the passenger name, flight name, destination, and travel date for all passengers travelling to Delhi.

Signature of the Course Faculty

Question 7: Library Management Database (10 Marks)

A library maintains the following database schema:

- Member (Member_ID, Member_Name, Phone)
- Book (Book_ID, Book_Title, Author)
- Issue (Issue_ID, Member_ID, Book_ID, Issue_Date)

Analyze the schema and write a Python program to retrieve the member name, book title, and issue date for all books issued after 1 January 2026.

Sqlite:

1. **Import the SqLite**

```
import sqlite3
```

2. **Connecting to SqLite**

```
conn = sqlite3.connect("library.db")

cur = conn.cursor()

if conn:

  print("SqLite Connected succesfully")

else:

  print("Not connected")
```

Output: SqLite Connected succesfully.

3. **Creating the Tables**

```
cur.execute("CREATE TABLE IF NOT EXISTS Member(Member_ID INTEGER PRIMARY KEY, Member_Name TEXT, Phone TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Book(Book_ID INTEGER PRIMARY KEY, Book_Title TEXT, Author TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Issue(Issue_ID INTEGER PRIMARY KEY, Member_ID INTEGER, Book_ID INTEGER, Issue_Date TEXT)")

print("Table Created Successfully")
```

Output: Tables Created Successfully

4. **Inserting the values to table**

```
cur.execute("INSERT INTO Member VALUES(1,'Rahul','9876543210')")

cur.execute("INSERT INTO Member VALUES(2,'Pawan','9494100715')")
```

```
cur.execute("INSERT INTO Member VALUES(3,'Ramesh','7569968275')")


cur.execute("INSERT INTO Book VALUES(101,'Python Basics','Sreleela')")
cur.execute("INSERT INTO Book VALUES(102,'C++ Basics','Anbazhgan')")
cur.execute("INSERT INTO Book VALUES(103,'C Basics','Thalapathy')")


cur.execute("INSERT INTO Issue VALUES(1,1,101,'2026-02-15')")
cur.execute("INSERT INTO Issue VALUES(2,2,102,'2026-02-15')")
cur.execute("INSERT INTO Issue VALUES(3,3,103,'2026-02-15')")


conn.commit()
print("Records Inserted Successfully")
```

Output: Records Created Successfully.

5. Printing the tables

```
cur.execute("SELECT * FROM Member")
rows = cur.fetchall()
print("Member Table")
for row in rows:
    print(row)
cur.execute("SELECT * FROM Book")
rows = cur.fetchall()
print("Book Table")
for row in rows:
  print(row)
cur.execute("SELECT * FROM Issue")
rows = cur.fetchall()
print("Book Table")
```

```
for row in rows:
  print(row)
```

Output:

Member Table

(1, 'Rahul', '9876543210')

(2, 'Pawan', '9494100715')

(3, 'Ramesh', '7569968275')

Book Table

(101, 'Python Basics', 'Sreleela')

(102, 'C++ Basics', 'Anbazhgan')

(103, 'C Basics', 'Thalapathy')

Book Table

(1, 1, 101, '2026-02-15')

(2, 2, 102, '2026-02-15')

(3, 3, 103, '2026-02-15')

6. **Read Operation**

```
cur.execute("""
SELECT Member.Member_Name,
Book.Book_Title,
Issue.Issue_Date
FROM Issue
JOIN Member
ON Issue.Member_ID=Member.Member_ID
JOIN Book
ON Issue.Book_ID=Book.Book_ID
WHERE Issue.Issue_Date>'2026-01-01'
""")
rows = cur.fetchall()
print("Library Details")
for row in rows:
   print(row)
```

Output:

Library Details

('Rahul', 'Python Basics', '2026-02-15')

('Pawan', 'C++ Basics', '2026-02-15')

('Ramesh', 'C Basics', '2026-02-15')

7. **Update Operation**

```
cur.execute("UPDATE Member SET Phone='7382882538' WHERE Member_ID=1")
conn.commit()
print("Record Updated Successfully"
cur.execute("SELECT * FROM Member")
rows = cur.fetchall()
print("Member Table")
for row in rows:
    print(row)
```

Output: Record updated Sucessfully

Member Table

(1, 'Rahul', '7382882538')

(2, 'Pawan', '9494100715')

(3, 'Ramesh', '7569968275')

8. **Delete Operation**

```
cur.execute("DELETE FROM Book WHERE Book_ID=101")
conn.commit()
print("Record Deleted Successfully")
cur.execute("SELECT * FROM Book")
rows = cur.fetchall()
print("Member Table")
for row in rows:
    print(row)
```

Output: Record Deleted Successfully

Book Table

(102, 'C++ Basics', 'Anbazhgan')

(103, 'C Basics', 'Thalapathy')

9. **Closing Connection**

```
Conn.close()
```

Question 8: Hotel Reservation Database (10 Marks)

A hotel maintains the following database schema:

- Customer (Customer_ID, Customer_Name, City)
- Room (Room_ID, Room_Type, Rent)
- Reservation (Reservation_ID, Customer_ID, Room_ID, Check_In_Date, Check_Out_Date)

Analyze the schema and write a Python program to retrieve the customer name, room type, and check-in date for customers who have reserved Deluxe rooms.

Sqlite:

1. **Import the SQLite Library**

```
import sqlite3
```

2. **Connecting to SQLite**

```
conn = sqlite3.connect("hotel.db")

cur = conn.cursor()

if conn:

   print("SQLite Connected Successfully")

else:

   print("Not Connected")
```

Output:

```
SQLite Connected Successfully
```

3. **Creating the Tables**

```
cur.execute("CREATE TABLE IF NOT EXISTS Customer(Customer_ID INTEGER PRIMARY
KEY, Customer_Name TEXT, City TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Room(Room_ID INTEGER PRIMARY KEY,
Room_Type TEXT, Rent INTEGER)")

cur.execute("CREATE TABLE IF NOT EXISTS Reservation(Reservation_ID INTEGER PRIMARY
KEY, Customer_ID INTEGER, Room_ID INTEGER, Check_In_Date TEXT, Check_Out_Date
TEXT)")

print("Tables Created Successfully")
```

Output:

```
Tables Created Successfully
```

4. Inserting the Values into the Tables (CREATE Operation)

```
cur.execute("INSERT INTO Customer VALUES(1,'Rahul','Chennai')")
cur.execute("INSERT INTO Customer VALUES(2,'Pawan','Hyderabad')")
cur.execute("INSERT INTO Customer VALUES(3,'Ramesh','Bangalore')")

cur.execute("INSERT INTO Room VALUES(101,'Deluxe',3000)")
cur.execute("INSERT INTO Room VALUES(102,'Standard',2000)")
cur.execute("INSERT INTO Room VALUES(103,'Suite',5000)")

cur.execute("INSERT INTO Reservation VALUES(1,1,101,'2026-03-10','2026-03-12')")
cur.execute("INSERT INTO Reservation VALUES(2,2,102,'2026-03-15','2026-03-17')")
cur.execute("INSERT INTO Reservation VALUES(3,3,103,'2026-03-20','2026-03-23')")
conn.commit()
print("Records Inserted Successfully")
```

Output:

```
Records Inserted Successfully
```

5. Printing the Tables

```
cur.execute("SELECT * FROM Customer")
rows = cur.fetchall()
print("Customer Table")
for row in rows:
    print(row)
cur.execute("SELECT * FROM Room")
rows = cur.fetchall()
print("Room Table")
for row in rows:
    print(row)
```

```
cur.execute("SELECT * FROM Reservation")

rows = cur.fetchall()

print("Reservation Table")

for row in rows:

    print(row)
```

Output:

```
Customer Table

(1, 'Rahul', 'Chennai')

(2, 'Pawan', 'Hyderabad')

(3, 'Ramesh', 'Bangalore')

Room Table

(101, 'Deluxe', 3000)

(102, 'Standard', 2000)

(103, 'Suite', 5000)

Reservation Table

(1, 1, 101, '2026-03-10', '2026-03-12')

(2, 2, 102, '2026-03-15', '2026-03-17')

(3, 3, 103, '2026-03-20', '2026-03-23')
```

6. Read Operation

```
cur.execute("""

SELECT Customer.Customer_Name,

Room.Room_Type,

Reservation.Check_In_Date

FROM Reservation

JOIN Customer

ON Reservation.Customer_ID=Customer.Customer_ID

JOIN Room
```

```
ON Reservation.Room_ID=Room.Room_ID

WHERE Room.Room_Type='Deluxe'

""")

rows = cur.fetchall()

print("Reservation Details")

for row in rows:

    print(row)
```

Output:

```
Reservation Details

('Rahul', 'Deluxe', '2026-03-10')
```

7. Update Operation

```
cur.execute("UPDATE Room SET Rent=3500 WHERE Room_ID=101")

conn.commit()

print("Record Updated Successfully")

cur.execute("SELECT * FROM Room")

rows = cur.fetchall()

print("Room Table")

for row in rows:

    print(row)
```

Output:

```
Record Updated Successfully

Room Table

(101, 'Deluxe', 3500)

(102, 'Standard', 2000)

(103, 'Suite', 5000)
```

8. Delete Operation

```
cur.execute("DELETE FROM Reservation WHERE Reservation_ID=1")
```

```
conn.commit()

print("Record Deleted Successfully")

cur.execute("SELECT * FROM Reservation")

rows = cur.fetchall()

print("Reservation Table")

for row in rows:

    print(row)
```

Output:

Record Deleted Successfully

Reservation Table

(2, 2, 102, '2026-03-15', '2026-03-17')

(3, 3, 103, '2026-03-20', '2026-03-23')

9. Closing the Connection

```
conn.close()

print("Connection Closed Successfully")
```

Output:

Connection Closed Successfully

https://colab.research.google.com/drive/1-N6ZYAbra_Byx12Ta7CmGJU3Lzks1Um9?usp=sharing